Workflow and architecture patterns for working together 2: Going forward

Jacob Archambault

June 7, 2026

Outline

1. Current challenges and their canonical solutions

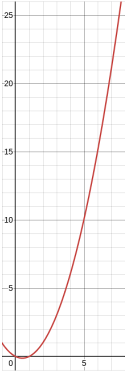
Outline

Outline

Outline

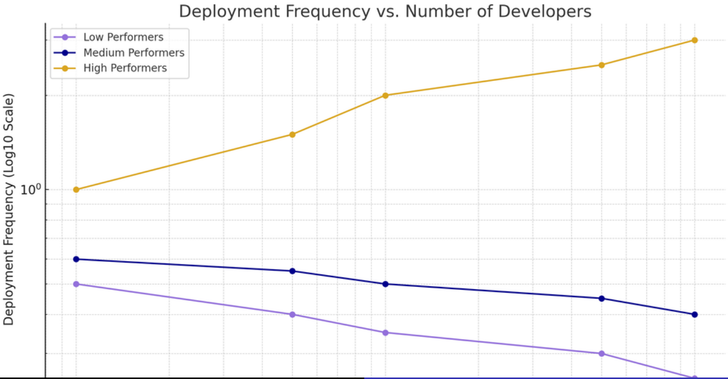
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals$=$ $n(n-1)/2$



The Mythical Man Month, Fred Brooks (cont.)

- corollary: adding more people to a project can lead not only to diminishing returns on delivery speed, but to objectively less work being completed



Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams
- late integration $\rightarrow$ adopting a true continuous integration workflow
- database coupling $\rightarrow$ Bezos API mandate/"Microservices"

1967: Conway's law

'Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.' - Melvin Conway, 'How do Committees Invent?' Datamation, 1967

Conway's law: examples

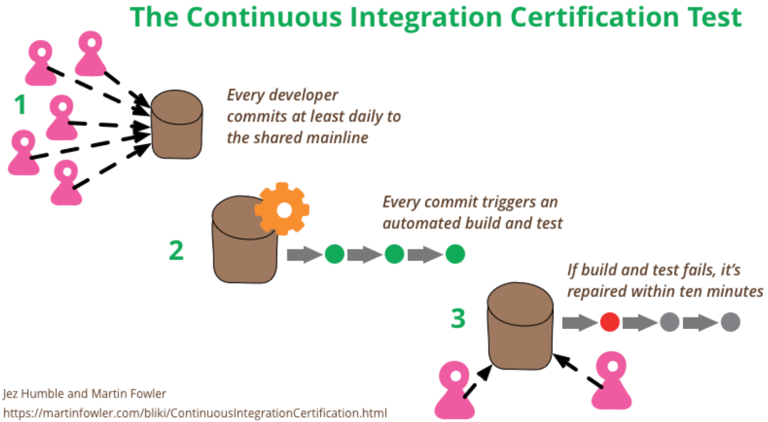
- separate `.csproj` projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
  - separate the codebases for improved velocity
  - join the teams for improved code sharing and communication (Inverse Conway maneuver)
  - branching-based strategies for team management compromise between the above approaches without their benefits

Outline

Defining continuous integration



Outline

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\textit{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(no\ conflicts) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(no\ conflicts) = \frac{1}{2^n}$ where $|C| = n$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\textit{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\textit{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
  - $P(\textit{no conflicts}) < 10\%$ where $n \geq 4$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\textit{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\textit{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
  - $P(\textit{no conflicts}) < 10\%$ where $n \geq 4$
  - $P(\textit{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\textit{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\textit{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
  - $P(\textit{no conflicts}) < 10\%$ where $n \geq 4$
  - $P(\textit{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A)) =$ probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(no\ conflicts(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
    - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
    - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
    - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
    - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
    - $= \frac{1}{32 - 2 - 4 - 4 + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
    - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
    - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
    - $= \frac{1}{32 - 2 - 4 - 4 + 3}$
    - $= \frac{1}{32 - 7}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(no\ conflicts(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
  - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
  - $= \frac{1}{32 - 2 - 4 - 4 + 3}$
  - $= \frac{1}{32 - 7}$
  - $= \frac{1}{25}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
    - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
    - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
    - $= \frac{1}{32 - 2 - 4 - 4 + 3}$
    - $= \frac{1}{32 - 7}$
    - $= \frac{1}{25}$
    - $= 4\%$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
  - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
  - $= \frac{1}{32 - 2 - 4 - 4 + 3}$
  - $= \frac{1}{32 - 7}$
  - $= \frac{1}{25}$
  - $= 4\%$

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then
$$P(no\ conflicts) = \frac{1}{2^n - (\sum_{i=1}^{k} 2^{n_i}) + k}$$

The State of DevOps report: branching findings

We found that having branches or forks with very short lifetimes (less than a day) before being merged into trunk, and less than three active branches in total, are important aspects of continuous delivery, and all contribute to higher performance. So does merging code into trunk or master on a daily basis.
— The 2016 State of DevOps Report, p. 31

Outline

The State of DevOps Report: throughput metrics

- lead time for changes

The State of DevOps Report: throughput metrics

- lead time for changes
- deployment frequency

The State of DevOps Report: stability metrics

- change failure rate

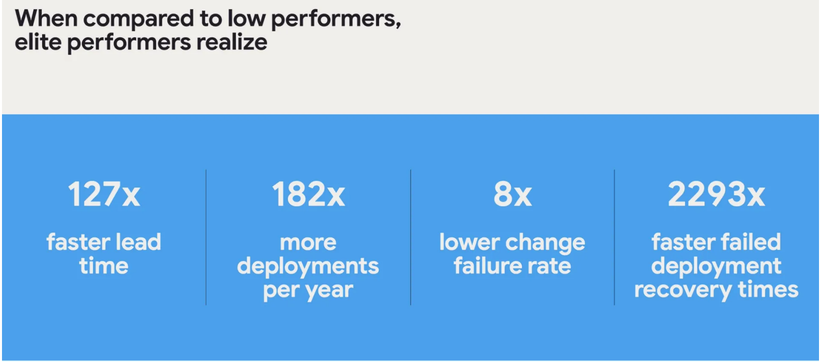
The State of DevOps Report: stability metrics

- change failure rate
- mean time to restore

The State of DevOps Report: results

'Astonishingly, these results demonstrate that there is no trade-off between improving performance and achieving higher levels of stability and quality. Rather, high performers do better at all of these measures. This is precisely what the Agile and Lean movements predict, but much dogma in our industry still rests on the false assumption that moving faster means trading off against other performance goals, rather than enabling and reinforcing them.' - Nicole Forsgren, Jez Humble, and Gene Kim, Accelerate (2017), p. 14

The State of DevOps Report: results



— 2024 State of DevOps Report

Approximating DevOps metrics in our current environment

- Deployment frequency $\rightarrow$ GitHub Actions run count

Approximating DevOps metrics in our current environment

- Deployment frequency $\rightarrow$ GitHub Actions run count
- Change failure rate $\rightarrow$ GitHub Actions pipeline failure rate

Approximating DevOps metrics in our current environment

- Deployment frequency $\rightarrow$ GitHub Actions run count
- Change failure rate $\rightarrow$ GitHub Actions pipeline failure rate
- Mean time to restore $\rightarrow$ percentage of merge commits to value-adding commits

Approximating DevOps metrics in our current environment

- Deployment frequency $\rightarrow$ GitHub Actions run count
- Change failure rate $\rightarrow$ GitHub Actions pipeline failure rate
- Mean time to restore $\rightarrow$ percentage of merge commits to value-adding commits

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar
- 60% the velocity of all other initiatives combined since 21 Jan

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar
- 60% the velocity of all other initiatives combined since 21 Jan

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr
- 158% better per contributor velocity than all other contributions combined since 23 Mar

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr
- 158% better per contributor velocity than all other contributions combined since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr
- 158% better unit test pass rate since 15 Apr

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr
- 158% better unit test pass rate since 15 Apr

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work
- 25% less backmerge work

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work
- 25% less backmerge work
- Source command: `git log --since="2026-04-15" --oneline <--merges> <--all> <^PEDI-35120_837D_COB> | <grep -iv ''merge pull request''> | wc -l`

Outline

Going forward: rollout

- Phase one focuses on a) greater integration between developers in the same repository, and b) greater integration of development and testing, with later phases targeted at improving dev/ops collaboration and intra-team CloudOps capabilities

Going forward: rollout

- Phase one focuses on a) greater integration between developers in the same repository, and b) greater integration of development and testing, with later phases targeted at improving dev/ops collaboration and intra-team CloudOps capabilities
- Strategy can be rolled out first for claims (837, 277) and authorizations (278), then other transactions as desired

Going forward: rollout

- Phase one focuses on a) greater integration between developers in the same repository, and b) greater integration of development and testing, with later phases targeted at improving dev/ops collaboration and intra-team CloudOps capabilities
- Strategy can be rolled out first for claims (837, 277) and authorizations (278), then other transactions as desired

Outline

Going forward: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
  - pull requests
  - testing
  - deployment

Going forward: team composition

- All teams working in same git repository are in the same daily scrum

Going forward: team composition

- All teams working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I will set this up.

Going forward: team composition

- All teams working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I will set this up.
- Teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)

Going forward: team composition

- All teams working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I will set this up.
- Teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- Teams can move work forward without waiting on or requiring outside assistance from another team

Going forward: team composition

- All teams working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I will set this up.
- Teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- Teams can move work forward without waiting on or requiring outside assistance from another team

Outline

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
    - Note: resist the temptation to branch off the shared branch

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
  - Note: resist the temptation to branch off the shared branch
- Branch is named for the primary Jira ticket being worked
- Every commit is pushed to origin only and immediately after passing local testing

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
  - Note: resist the temptation to branch off the shared branch
- Branch is named for the primary Jira ticket being worked
- Every commit is pushed to origin only and immediately after passing local testing
- Anyone can revert a bad commit

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
    - Note: resist the temptation to branch off the shared branch
- Branch is named for the primary Jira ticket being worked
- Every commit is pushed to origin only and immediately after passing local testing
- Anyone can revert a bad commit
- Every git commit message begins with a Jira ticket number

Going forward: branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
  - Note: resist the temptation to branch off the shared branch
- Branch is named for the primary Jira ticket being worked
- Every commit is pushed to origin only and immediately after passing local testing
- Anyone can revert a bad commit
- Every git commit message begins with a Jira ticket number

Outline

Going forward: testing

- Every change is tested by the test suite locally before committing

Going forward: testing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written

Going forward: testing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.

Going forward: testing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component

Going forward: testing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component
- Goal is to move most testing into the test suite so that manual checks become minimal.

Going forward: testing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component
- Goal is to move most testing into the test suite so that manual checks become minimal.

Outline

Going forward: pull requests

- Pull requests occur at least daily

Going forward: pull requests

- Pull requests occur at least daily
- Because the working branch is by hypothesis always releasable, any contributor can PR the working branch at any time in principle. In practice, this will be worked out informally beforehand.

Going forward: pull requests

- Pull requests occur at least daily
- Because the working branch is by hypothesis always releasable, any contributor can PR the working branch at any time in principle. In practice, this will be worked out informally beforehand.
- To allow time for review before end of day, pull requests should generally be set up by 3-3:30pm eastern time.

Going forward: pull requests

- Pull requests occur at least daily
- Because the working branch is by hypothesis always releasable, any contributor can PR the working branch at any time in principle. In practice, this will be worked out informally beforehand.
- To allow time for review before end of day, pull requests should generally be set up by 3-3:30pm eastern time.
- Because leads are also both contributing to and seeing changes to the working branch in real time, in practice PR's becomes increasingly *pro forma*

Going forward: pull requests

- Pull requests occur at least daily
- Because the working branch is by hypothesis always releasable, any contributor can PR the working branch at any time in principle. In practice, this will be worked out informally beforehand.
- To allow time for review before end of day, pull requests should generally be set up by 3-3:30pm eastern time.
- Because leads are also both contributing to and seeing changes to the working branch in real time, in practice PR's becomes increasingly *pro forma*

Going forward: branching strategies for frequent pull requests

- Continuing branch, e.g., PEDI-XXXX-feature_description

Going forward: branching strategies for frequent pull requests

- Continuing branch, e.g., PEDI-XXXX-feature_description
  - Branch created, PR created, branch merged, develop merged into PR branch, work continues until feature is complete

Going forward: branching strategies for frequent pull requests

- Continuing branch, e.g., PEDI-XXXX-feature_description
  - Branch created, PR created, branch merged, develop merged into PR branch, work continues until feature is complete
- Stacked daily branches, e.g., PEDI-XXXX-feature_description-YYYYMMDD

Going forward: branching strategies for frequent pull requests

- Continuing branch, e.g., PEDI-XXXX-feature_description
    - Branch created, PR created, branch merged, develop merged into PR branch, work continues until feature is complete
- Stacked daily branches, e.g., PEDI-XXXX-feature_description-YYYYMMDD
    - Branch created, PR created, new branch created off of PR branch, work continues, backmerge develop as needed

Going forward: branching strategies for frequent pull requests

- Continuing branch, e.g., PEDI-XXXX-feature_description
    - Branch created, PR created, branch merged, develop merged into PR branch, work continues until feature is complete
- Stacked daily branches, e.g., PEDI-XXXX-feature_description-YYYYMMDD
    - Branch created, PR created, new branch created off of PR branch, work continues, backmerge develop as needed

Outline

Going forward: continuous integration enablers

- improving CI pipeline runtime

Going forward: continuous integration enablers

- improving CI pipeline runtime
- team discipline

Going forward: continuous integration enablers

- improving CI pipeline runtime
- team discipline
- fast PR turnaround

Going forward: continuous integration enablers

- improving CI pipeline runtime
- team discipline
- fast PR turnaround

Going forward: enabling teams

- *Enabling team* - helps a stream-aligned team to overcome obstacles. Also detects missing capabilities.

Going forward: enabling teams

- *Enabling team* - helps a stream-aligned team to overcome obstacles. Also detects missing capabilities.
- I will marshal new onboarding team members into a makeshift enabling team for improving our test suite speed. This also helps them to begin contributing earlier in a low-risk way.

Going forward: enabling teams

- *Enabling team* - helps a stream-aligned team to overcome obstacles. Also detects missing capabilities.
- I will marshal new onboarding team members into a makeshift enabling team for improving our test suite speed. This also helps them to begin contributing earlier in a low-risk way.
- The 837 COB team will serve as an enabling team for piloting a teams-based branching strategy in the claims (837 and 277) and auth (278) repos

Going forward: enabling teams

- *Enabling team* - helps a stream-aligned team to overcome obstacles. Also detects missing capabilities.
- I will marshal new onboarding team members into a makeshift enabling team for improving our test suite speed. This also helps them to begin contributing earlier in a low-risk way.
- The 837 COB team will serve as an enabling team for piloting a teams-based branching strategy in the claims (837 and 277) and auth (278) repos

Outline

Going forward: getting to square one

- James, Peter, and the 837 COB team can negotiate merge order for remaining two 837 initiatives

Going forward: getting to square one

- James, Peter, and the 837 COB team can negotiate merge order for remaining two 837 initiatives
- The next of these initiatives merges to develop as soon as possible

Going forward: getting to square one

- James, Peter, and the 837 COB team can negotiate merge order for remaining two 837 initiatives
- The next of these initiatives merges to develop as soon as possible
- Members on the previous initiatives then move onto the 837 COB team branch to a) help with merge conflict resolution, and b) refactor to allow for easily backing out of any feature should the business decide against releasing a particular feature or if they change the order.

Going forward: getting to square one

- James, Peter, and the 837 COB team can negotiate merge order for remaining two 837 initiatives
- The next of these initiatives merges to develop as soon as possible
- Members on the previous initiatives then move onto the 837 COB team branch to a) help with merge conflict resolution, and b) refactor to allow for easily backing out of any feature should the business decide against releasing a particular feature or if they change the order.
    - Key is to achieve isolation through architecture rather than through branching

Going forward: getting to square one

- James, Peter, and the 837 COB team can negotiate merge order for remaining two 837 initiatives
- The next of these initiatives merges to develop as soon as possible
- Members on the previous initiatives then move onto the 837 COB team branch to a) help with merge conflict resolution, and b) refactor to allow for easily backing out of any feature should the business decide against releasing a particular feature or if they change the order.
  - Key is to achieve isolation through architecture rather than through branching

Workflow and architecture patterns for working together 2: Going forward

Jacob Archambault

June 7, 2026